

NLP Tutor

De Qwen2.5 à un assistant pédagogique spécialisé avec QLoRA

Wendpêgré Hugues Anselme KABORE

30 septembre 2026

Table des matières

0.1	Comment lire ce rapport	2
0.2	1. Objectif du projet	3
1	2. Organisation du projet	3
2	3. config.py — configuration générale	5
3	4. model.py — chargement du tokenizer et des modèles	5
3.1	4.1 Tokenizer	5
4	5. Comprendre tokenisation, logits et softmax	6
4.1	5.1 Tokenisation	6
4.2	5.2 Logits	6
4.3	5.3 Softmax	6
5	6. Stratégies de génération	7
5.1	Greedy decoding	7
5.2	Température	7
5.3	Top-k	8
5.4	Top-p	8
6	7. generation.py — génération des réponses	8
7	8. Baseline — modèle sans fine-tuning	9
8	9. dataset.py — construction du dataset pédagogique	9
8.1	Limite importante	10
9	10. Pourquoi séparer train, validation et test ?	10
10	11. training.py — adaptation QLoRA	11
10.1	11.1 Quantification 4 bits	11
10.2	11.2 Préparation du modèle	11
10.3	11.3 LoRA	11
11	12. Paramètres réellement entraînés	12
12	13. Configuration du SFTTrainer	12
13	14. Entraînement	13
14	15. Sauvegarde de l'adaptateur	14
15	16. evaluation.py — métriques	14
15.1	16.1 ROUGE-L	15
15.2	16.2 Couverture des mots-clés	15

1617. Évaluation de la baseline	15
1718. Évaluation après QLoRA	16
1819. Pourquoi ROUGE-L = 1 ne veut pas dire « modèle parfait »	16
1920. Challenge de généralisation	17
2021. Exemple de limite observée	17
2122. app/app.py — interface Gradio	18
2223. Limite actuelle de l'historique conversationnel	19
2324. Résultats enregistrés	19
2425. Pourquoi QLoRA était adapté à ce projet	20
2526. Technologies utilisées	20
2627. Ce que chaque fichier apporte	21
26.1 config.py	21
26.2 model.py	21
26.3 generation.py	21
26.4 dataset.py	21
26.5 training.py	22
26.6 evaluation.py	22
26.7 app/app.py	22
26.8 notebooks/agent.ipynb	22
2728. Difficultés rencontrées pendant le développement	22
27.1 Environnement Colab / VS Code	23
27.2 Imports du package	23
27.3 Dépendances	23
27.4 Mémoire GPU	23
27.5 Sauvegarde	23
2829. Limites du projet	23
2930. Améliorations possibles	23
29.1 Données	23
29.2 Évaluation	24
29.3 Conversation	24
29.4 Déploiement	24
3031. Conclusion	24
3132. Résumé en une phrase	25

0.1 Comment lire ce rapport

Ce document est volontairement présenté dans un style **R Markdown / Quarto**, mais le projet lui-même est écrit en **Python**. Les blocs de code montrent les instructions réellement utilisées ou leur version directement équivalente dans les modules du projet. Les blocs text montrent des sorties, des structures de dossiers ou des résultats observés.

L'objectif n'est pas seulement de présenter les scores finaux : chaque grande étape explique **ce qui a été fait, dans quel fichier, pourquoi ce choix a été fait, et comment le résultat doit être interprété**.

0.2 1. Objectif du projet

L'objectif était de construire un assistant pédagogique capable de répondre en français à des questions sur les notions fondamentales du NLP et des Transformers.

Le projet compare deux configurations :

1. **Baseline** : le modèle Qwen2.5-1.5B-Instruct utilisé directement avec un prompt système, sans modifier ses poids ;
2. **QLoRA** : le même modèle, quantifié en 4 bits, auquel on ajoute des adaptateurs LoRA entraînés sur un petit dataset pédagogique.

L'idée n'était donc pas de construire un assistant généraliste, mais un système spécialisé, mesurable et suffisamment léger pour être entraîné sur un GPU Tesla T4.

Le pipeline global est :

```
Question utilisateur
↓
Tokenizer
↓
IDs de tokens
↓
Transformer Qwen2.5
↓
Logits
↓
Softmax / stratégie de décodage
↓
Réponse générée
↓
Évaluation
↓
Interface Gradio
```

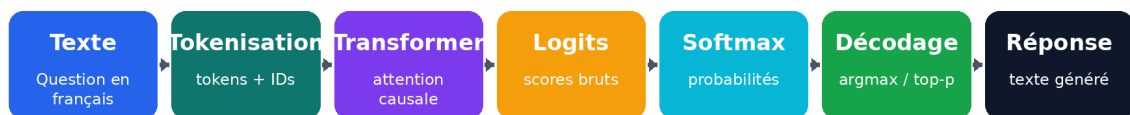


Figure 1: Vue synthétique du pipeline NLP Tutor

Figure : chaîne expérimentale depuis la requête utilisateur jusqu'à l'évaluation et l'interface.

1 2. Organisation du projet

Le code a été séparé en plusieurs fichiers afin d'éviter de mettre toute la logique dans le notebook.

```
nlp_tutor/
├── app/
│   └── app.py
├── data/
│   ├── concepts.json
│   └── question_templates.json
└── notebooks/
```

```

├── agent.ipynb
├── results/
│   ├── baseline_results.csv
│   ├── qlora_results.csv
│   ├── baseline_vs_qlora.csv
│   └── challenge_comparison.csv
├── src/
│   └── nlp_tutor/
│       ├── __init__.py
│       ├── config.py
│       ├── dataset.py
│       ├── evaluation.py
│       ├── generation.py
│       ├── model.py
│       └── training.py
├── pyproject.toml
├── requirements.txt
└── README.md

```

Cette architecture permet de distinguer :

- la **configuration** ;
- le **chargement des modèles** ;
- la **génération** ;
- la **construction du dataset** ;
- l'**évaluation** ;
- le **fine-tuning** ;
- l'**interface applicative**.

Architecture logicielle de NLP Tutor

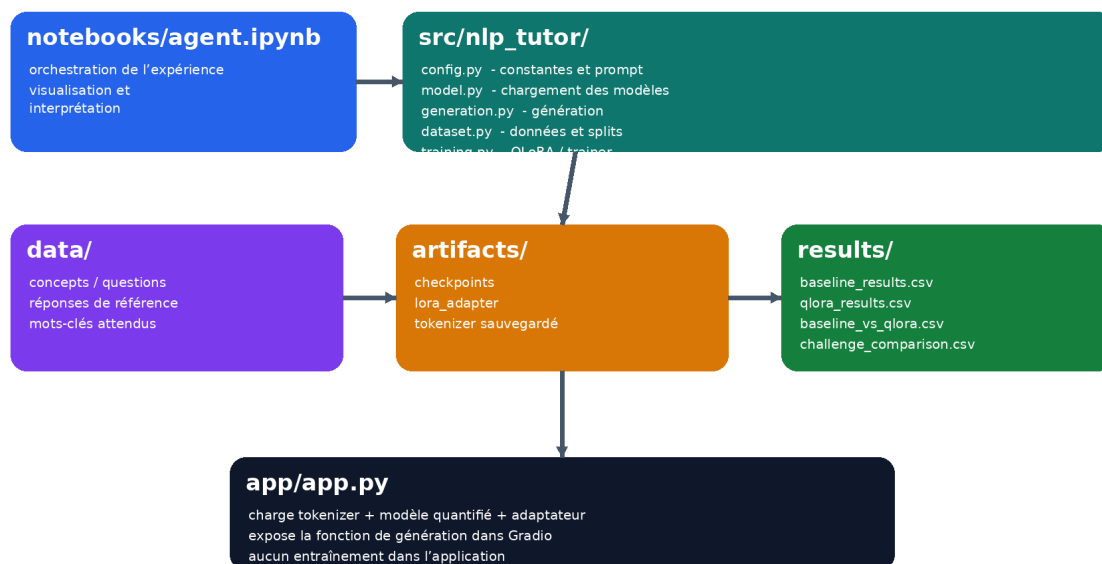


Figure 2: Architecture logicielle du projet

Cette séparation rend le code plus lisible et surtout réutilisable : le notebook ne porte plus toute la logique métier, il orchestre des fonctions définies dans `src/nlp_tutor/`.

Le notebook `agent.ipynb` sert surtout à orchestrer les expériences et à afficher les résultats.

2 3. config.py — configuration générale

Le fichier config.py centralise les constantes communes au projet.

Exemple :

```
MODEL_ID = "Qwen/Qwen2.5-1.5B-Instruct"
SEED = 42
```

La graine aléatoire permet d'améliorer la reproductibilité :

```
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

Le prompt système définit le comportement pédagogique attendu :

```
SYSTEM_PROMPT = (
    "Tu es NLP Tutor, un assistant pédagogique francophone spécialisé en NLP, "
    "modèles de langage et Transformers.\n"
    "Règles :\n"
    "1. Réponds en français.\n"
    "2. Commence par une définition claire.\n"
    "3. Explique progressivement avec des mots simples.\n"
    "4. Donne au moins un exemple concret quand c'est pertinent.\n"
    "5. Termine par une phrase 'À retenir : ...'.\n"
    "6. N'invente pas une information si tu n'es pas sûr."
)
```

Ce prompt sert à donner une identité stable au système avant même le fine-tuning.

3 4. model.py — chargement du tokenizer et des modèles

Ce fichier regroupe la logique de chargement.

Les fonctions principales sont :

```
load_tokenizer()
load_base_model()
load_qlora_base_model()
load_finetuned_model()
```

3.1 4.1 Tokenizer

Le tokenizer convertit le texte humain en identifiants numériques.

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

tokenizer.padding_side = "right"
```

Exemple observé pendant l'expérience :

Texte : "Un modèle de langage prédit le prochain token."

```
Nombre de mots   : 8
Nombre de tokens : 13
```

Cela montre qu'un token n'est pas nécessairement un mot complet.
Un mot peut être découpé en plusieurs sous-unités.

4 5. Comprendre tokenisation, logits et softmax

Avant de fine-tuner le modèle, le notebook étudie son fonctionnement interne.

4.1 5.1 Tokenisation

Extrait du notebook :

```
texte = "Un modèle de langage prédit le prochain token."

encodage = tokenizer(
    texte,
    add_special_tokens=False,
)

token_ids = encodage["input_ids"]
```

Pour visualiser les tokens :

```
df_tokens = pd.DataFrame({
    "position": range(len(token_ids)),
    "token_id": token_ids,
    "token_interne": tokenizer.convert_ids_to_tokens(token_ids),
    "morceau_decode": [
        tokenizer.decode([i])
        for i in token_ids
    ],
})
```

Cette étape permet de relier le texte à la représentation numérique réellement manipulée par le modèle.

4.2 5.2 Logits

Pour une séquence donnée, le Transformer renvoie des logits.

```
with torch.inference_mode():
    outputs = base_model(**inputs)

next_logits = outputs.logits[0, -1]
```

La forme générale de `outputs.logits` est :

```
[batch_size, longueur_sequence, taille_vocabulaire]
```

On prend la dernière position parce que l'objectif est de prédire le prochain token.

4.3 5.3 Softmax

Les logits sont des scores bruts.

Ils ne sont pas encore des probabilités.

La softmax les transforme en distribution probabiliste :

$$P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

En Python :

```
next_probs = torch.softmax(
    next_logits.float(),
    dim=-1,
)
```

Puis :

```
argmax_id = torch.argmax(next_probs).item()
```

argmax choisit le token de probabilité maximale.

5 6. Stratégies de génération

Le projet compare plusieurs stratégies.

```
strategies = [
    ("greedy", dict(do_sample=False)),
    (
        "temperature_0.7",
        dict(
            do_sample=True,
            temperature=0.7,
        ),
    ),
    (
        "top_k_20",
        dict(
            do_sample=True,
            temperature=0.8,
            top_k=20,
        ),
    ),
    (
        "top_p_0.9",
        dict(
            do_sample=True,
            temperature=0.8,
            top_p=0.9,
        ),
    ),
]
```

5.1 Greedy decoding

```
do_sample=False
```

Le modèle sélectionne le token ayant la probabilité maximale.
C'est déterministe si le reste de l'environnement est déterministe.

5.2 Température

La température modifie la distribution :

$$P_i(T) = \text{softmax} \left(\frac{z_i}{T} \right)$$

Une température faible concentre la distribution.

Une température plus élevée rend les alternatives plus probables et augmente la diversité.

5.3 Top-k

Le top-k garde seulement les k tokens les plus probables.

```
top_k=20
```

5.4 Top-p

Le top-p garde le plus petit ensemble de tokens dont la probabilité cumulée atteint p.

```
top_p=0.9
```

Le nombre de candidats varie donc selon la distribution.

6 7. generation.py — génération des réponses

La fonction principale utilisée dans le projet est conceptuellement :

```
def generate_tutor(
    question,
    model,
    tokenizer,
    do_sample=False,
    temperature=0.4,
    top_p=0.9,
    max_new_tokens=220,
):
    ...
```

La question est transformée en conversation :

```
messages = [
    {
        "role": "system",
        "content": SYSTEM_PROMPT,
    },
    {
        "role": "user",
        "content": question,
    },
]
```

Puis le chat template du tokenizer Qwen est utilisé :

```
model_inputs = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    tokenize=True,
    return_dict=True,
    return_tensors="pt",
).to(model.device)
```

Ensuite le modèle génère les nouveaux tokens :


```

outputs = model.generate(
    **model_inputs,
    max_new_tokens=max_new_tokens,
    do_sample=do_sample,
    temperature=temperature,
    top_p=top_p,
)

```

Enfin, seule la partie nouvellement générée est décodée.

Cette fonction est utilisée à la fois pour la baseline et pour le modèle QLoRA afin de comparer les deux systèmes de manière cohérente.

7 8. Baseline — modèle sans fine-tuning

La baseline repose sur :

```

Qwen2.5-1.5B-Instruct
+
SYSTEM_PROMPT
+
paramètres de génération

```

Aucun poids du modèle n'est modifié.

Exemples de questions :

```

baseline_questions = [
    "Qu'est-ce qu'un token en NLP ?",
    "À quoi sert l'attention dans un Transformer ?",
    "Quelle est la différence entre fine-tuning et prompting ?",
]

```

Cette première étape permet de répondre à la question :

Que sait déjà faire le modèle uniquement grâce à son pré-entraînement et à l'instruction ?

La baseline est ensuite conservée comme point de comparaison.

8 9. dataset.py — construction du dataset pédagogique

Le dataset comprend :

```

25 concepts
×
20 formulations
=
500 couples question-réponse

```

Répartition :

```

Train      : 350
Validation : 75
Test       : 75

```

Pour chaque concept :

```

14 formulations → train
3 formulations  → validation
3 formulations  → test

```

Exemple de concept :

```
"softmax": (  
    "La softmax transforme un vecteur de scores en probabilités positives "  
    "dont la somme vaut 1. À retenir : elle convertit les logits en "  
    "distribution probabiliste.",  
    [  
        "probabilités",  
        "somme",  
        "logits",  
        "distribution",  
    ],  
)
```

Les templates de questions introduisent différentes formulations :

```
QUESTION_TEMPLATES = [  
    "Explique-moi simplement {concept}.",  
    "C'est quoi {concept} en NLP ?",  
    "Peux-tu définir {concept} ?",  
    "À quoi sert {concept} ?",  
    "Comment expliquer {concept} à un débutant ?",  
    "Explique {concept} avec un exemple.",  
    ...  
]
```

8.1 Limite importante

Même si les questions changent, la réponse cible associée à un concept reste très similaire. Le modèle peut donc apprendre une association très directe :

```
concept  
→  
réponse pédagogique type
```

C'est une limite essentielle lors de l'interprétation des résultats.

9 10. Pourquoi séparer train, validation et test ?

Les trois ensembles ont des rôles différents.

```
TRAIN  
→ apprendre les paramètres  
  
VALIDATION  
→ suivre le comportement pendant le développement  
  
TEST  
→ évaluation finale
```

Le modèle n'est pas entraîné sur les données de test.

Dans le projet, 20 requêtes sont sélectionnées de manière reproductible dans le test :

```
eval20 = (  
    dataset["test"]  
    .shuffle(seed=123)  
    .select(range(20))  
)
```

Cette sélection sert exactement de la même manière pour la baseline et QLoRA.

10 11. training.py — adaptation QLoRA

Le but était de spécialiser Qwen sans réentraîner l'ensemble de ses 1,56 milliard de paramètres.

10.1 11.1 Quantification 4 bits

Configuration :

```
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
)
```

Le modèle est ensuite chargé en 4 bits :

```
qlora_model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    quantization_config=quantization_config,
    dtype=torch.float16,
    device_map={"": 0},
)
```

La quantification réduit fortement la mémoire nécessaire.

10.2 11.2 Préparation du modèle

```
qlora_model.config.use_cache = False

qlora_model = prepare_model_for_kbit_training(
    qlora_model
)
```

Le cache est désactivé pendant l'entraînement afin d'être compatible avec le gradient checkpointing.

10.3 11.3 LoRA

Configuration utilisée :

```
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",

    target_modules=[
        "q_proj",
        "k_proj",
        "v_proj",
        "o_proj",
        "gate_proj",
        "up_proj",
        "down_proj",
    ],
)
```

r=16 représente le rang des matrices LoRA.

LoRA ajoute de petites matrices entraînables alors que les poids principaux du modèle restent gelés.

Les modules ciblés couvrent :

- les projections de l'attention ;
- certaines projections du MLP.

11 12. Paramètres réellement entraînés

Le trainer affiche :

```
Paramètres entraînables : 18 464 768
Paramètres totaux       : 1 562 179 072
Part entraînable        : 1,182 %
```

Donc environ :

98,818 % des paramètres

ne sont pas entraînés directement.

C'est l'intérêt principal de LoRA / QLoRA : réduire fortement le coût de l'adaptation.

LoRA : petite fraction des poids entraînée

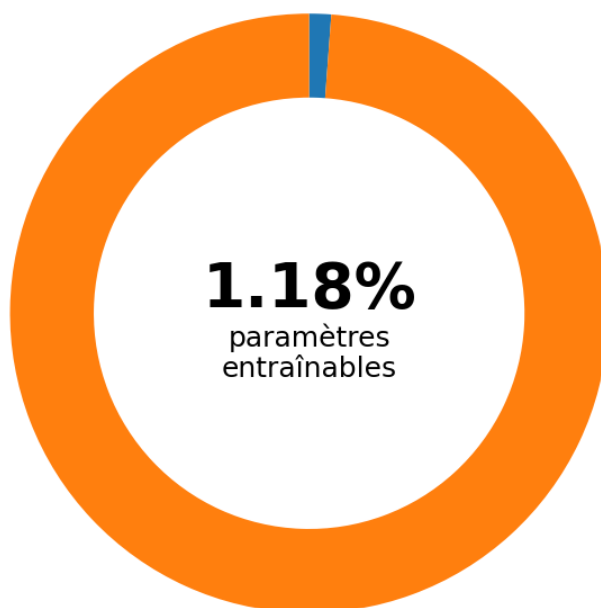


Figure 3: Part des paramètres entraînables avec LoRA

La très grande majorité des poids du modèle reste gelée ; seuls les adaptateurs sont optimisés.

12 13. Configuration du SFTTrainer

Le fine-tuning est réalisé avec TRL.

```

training_args = SFTConfig(
    output_dir=OUTPUT_DIR,

    num_train_epochs=2,

    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,

    gradient_accumulation_steps=8,

    learning_rate=2e-4,
    warmup_steps=10,

    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="epoch",
    save_total_limit=2,

    fp16=False,
    bf16=False,

    gradient_checkpointing=True,

    max_length=512,

    completion_only_loss=True,

    optim="adamw_torch",
    report_to="none",
    seed=SEED,
)

```

Le batch effectif vaut :

$2 \times 8 = 16$ exemples

`completion_only_loss=True` signifie que la loss porte sur la réponse attendue, pas sur le prompt utilisateur lui-même.

Le trainer est construit ainsi :

```

trainer = SFTTrainer(
    model=qlora_model,
    args=training_args,

    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"],

    processing_class=tokenizer,
    peft_config=lora_config,
)

```

13 14. Entraînement

Le lancement est simple :

```
train_result = trainer.train()
```

Le run effectué comprenait :

2 epochs
44 steps

GPU : Tesla T4

Une sortie observée lors du run donne notamment :

```
training_loss ≈ 0.717
```

La validation finale a donné environ :

```
eval_loss      ≈ 0.0143
perplexité     ≈ 1.014
mean token accuracy ≈ 0.9988
```

Ces valeurs indiquent que le modèle reproduit très bien les réponses attendues sur les formulations proches du dataset.

Cela ne prouve cependant pas une généralisation parfaite.

14 15. Sauvegarde de l'adaptateur

À la fin de l'entraînement :

```
eval_metrics = trainer.evaluate()
```

Puis :

```
ADAPTER_DIR = (
    PROJECT_ROOT
    / "artifacts"
    / "lora_adapter"
)

ADAPTER_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

trainer.model.save_pretrained(
    ADAPTER_DIR
)

tokenizer.save_pretrained(
    ADAPTER_DIR
)
```

L'intérêt est important :

Il n'est pas nécessaire de refaire l'entraînement pour la démonstration.

On peut recharger :

```
Qwen2.5-1.5B-Instruct
+
adaptateur LoRA sauvegardé
```

15 16. evaluation.py — métriques

Deux métriques principales sont utilisées.

15.1 16.1 ROUGE-L

Le code utilise :

```
from rouge_score import rouge_scorer

rouge = rouge_scorer.RougeScorer(
    ["rougeL"],
    use_stemmer=False,
)
```

Puis :

```
rouge_l = rouge.score(
    reference,
    prediction,
)["rougeL"].fmeasure
```

ROUGE-L mesure le recouvrement basé sur la plus longue sous-séquence commune entre la réponse générée et la référence.

Un score proche de 1 signifie donc une forte proximité textuelle.

Il ne signifie pas automatiquement :

"réponse parfaite"

15.2 16.2 Couverture des mots-clés

Chaque concept contient quelques mots-clés attendus.

Exemple :

```
[
    "probabilités",
    "somme",
    "logits",
    "distribution",
]
```

La fonction mesure la proportion de ces mots-clés présents dans la prédiction.

Conceptuellement :

```
coverage = (
    nombre_de_mots_cles_presentes
    /
    nombre_total_de_mots_cles
)
```

Cette métrique complète ROUGE-L en vérifiant la présence du vocabulaire essentiel.

16 17. Évaluation de la baseline

Le modèle de base est testé sur les mêmes 20 exemples.

Résultats :

Configuration	ROUGE-L moyen	Couverture mots-clés
Baseline + prompting	0,1331	0,2500

Ces résultats montrent que le modèle généraliste sait souvent parler des notions étudiées, mais

qu'il ne reproduit pas nécessairement le format pédagogique des références.

17 18. Évaluation après QLoRA

Le même protocole est exécuté après adaptation.

```
finetuned_results = evaluate_dataset(  
    model=finetuned_model,  
    tokenizer=tokenizer,  
    eval_dataset=eval20,  
    model_name="Qwen2.5-1.5B + QLoRA",  
)
```

Résultats :

Configuration	ROUGE-L moyen	Couverture mots-clés
Baseline + prompting	0,1331	0,2500
Après QLoRA	1,0000	0,8375

Évolution :

Δ ROUGE-L $\approx +0,8669$

Δ couverture $\approx +0,5875$

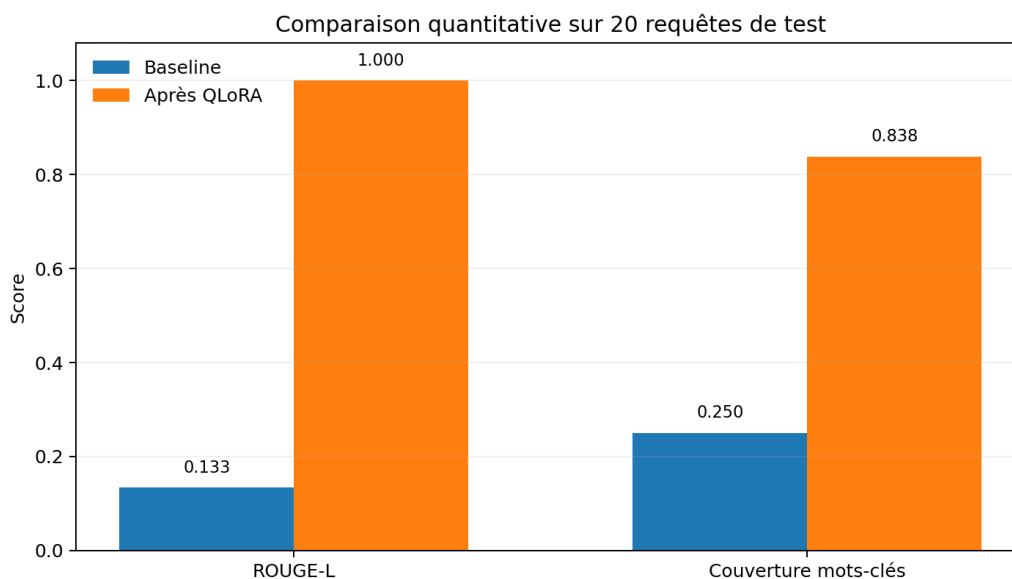


Figure 4: Comparaison quantitative entre la baseline et QLoRA

Le gain automatique est très important sur le test proche du dataset, mais doit être interprété avec prudence.

18 19. Pourquoi ROUGE-L = 1 ne veut pas dire « modèle parfait »

C'est l'un des points les plus importants du projet.

Le dataset contient différentes formulations d'une même notion, mais la réponse cible reste souvent identique ou presque identique.

Le modèle peut donc apprendre très fortement :

```
notion détectée  
→  
réponse type mémorisée
```

Ainsi :

```
ROUGE-L = 1
```

signifie surtout que, sur les 20 exemples évalués, la sortie correspond extrêmement bien à la réponse de référence.

Cela ne permet pas de conclure que le modèle saura répondre à toutes les nouvelles questions possibles.

19 20. Challenge de généralisation

Pour tester une situation plus réaliste, un second ensemble de 20 questions différentes a été créé.

Exemples :

```
Pourquoi un même mot peut-il être découpé différemment  
selon le tokenizer ?
```

```
Quelle différence y a-t-il entre les logits et  
les probabilités finales ?
```

```
Pourquoi une température très élevée peut-elle  
dégrader une réponse ?
```

```
Dans quel cas top-p est-il préférable à top-k ?
```

```
Pourquoi un Transformer a-t-il besoin  
d'informations de position ?
```

Ces questions demandent davantage de comparaison, de justification ou de raisonnement.

Le fichier :

```
results/challenge_comparison.csv
```

conserve côte à côte :

```
question  
baseline  
qlora
```

Cette seconde évaluation est importante car elle met en évidence les limites que les métriques automatiques ne capturent pas toujours.

20 21. Exemple de limite observée

Sur certaines questions nouvelles, QLoRA produit une réponse courte et très proche d'une définition apprise, mais peut ne pas traiter entièrement la demande.

Cela montre qu'une spécialisation trop étroite peut produire un comportement plus rigide.

Le modèle devient meilleur sur son format cible, mais il ne devient pas automatiquement

meilleur sur tous les types de raisonnement.

21 22. app/app.py — interface Gradio

L'interface sert uniquement à l'inférence.

Elle ne relance pas l'entraînement.

Le démarrage charge :

```
tokenizer
+
Qwen quantifié
+
adaptateur LoRA
```

Puis :

```
def answer_question(
    message,
    history,
):
    if message is None:
        return "Veuillez saisir une question."

    message = message.strip()

    if not message:
        return "Veuillez saisir une question."

    response = generate_tutor(
        message,
        MODEL,
        TOKENIZER,
        do_sample=True,
        temperature=0.4,
        top_p=0.9,
        max_new_tokens=250,
    )

    return response
```

L'interface :

```
demo = gr.ChatInterface(
    fn=answer_question,

    title="NLP Tutor",

    description=(
        "Assistant pédagogique francophone "
        "spécialisé dans le NLP, "
        "les modèles de langage et les Transformers."
    ),

    examples=[
        "Qu'est-ce qu'un token en NLP ?",
        "Explique-moi simplement la self-attention.",
        "Quelle est la différence entre LoRA et QLoRA ?",
        "À quoi sert la température pendant la génération ?",
    ],
)
```

Puis :

```
demo.launch(  
    share=True,  
    show_error=True,  
)
```

Cela crée un lien temporaire `gradio.live` pour la démonstration.

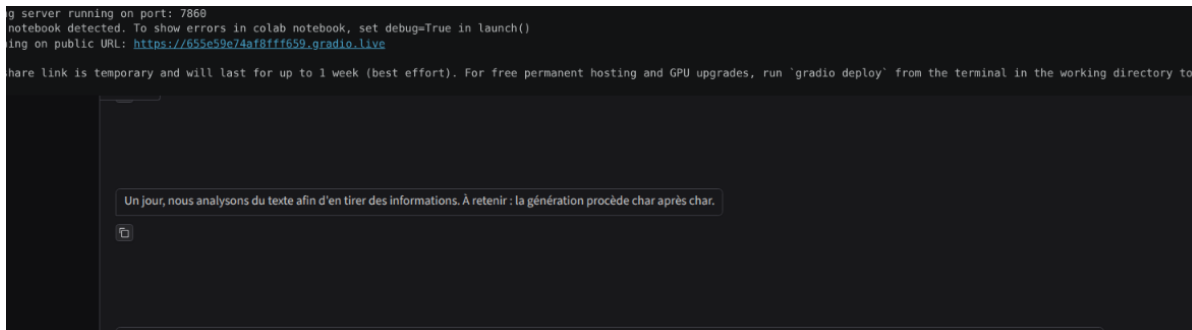


Figure 5: Interface Gradio de NLP Tutor pendant la démonstration

L'application recharge le modèle de base et l'adaptateur sauvegardé ; elle ne relance pas l'entraînement.

22 23. Limite actuelle de l'historique conversationnel

Gradio transmet bien un paramètre `history`.

Cependant, dans la version actuelle :

```
def answer_question(message, history):
```

`history` n'est pas réinjecté dans `generate_tutor`.

Le modèle répond donc principalement à la question courante.

Autrement dit, l'interface est conversationnelle visuellement, mais le système n'exploite pas encore réellement la mémoire multi-tour.

Une amélioration future serait :

```
history  
+  
nouvelle question  
↓  
construction du contexte limité  
↓  
chat template  
↓  
génération
```

avec une gestion explicite de la fenêtre de contexte.

23 24. Résultats enregistrés

Le projet sauvegarde les sorties dans `results/`.

Exemples :

```
baseline_results.csv
qlora_results.csv
baseline_vs_qlora.csv
challenge_comparison.csv
```

Cela permet de préparer la présentation sans réexécuter l'ensemble de l'expérience.

On peut charger directement :

```
baseline = pd.read_csv(
    "results/baseline_results.csv"
)

qlora = pd.read_csv(
    "results/qlora_results.csv"
)

comparison = pd.read_csv(
    "results/baseline_vs_qlora.csv"
)
```

24 25. Pourquoi QLoRA était adapté à ce projet

Un fine-tuning complet aurait nécessité d'optimiser environ 1,56 milliard de paramètres.

Avec QLoRA :

modèle principal
→ quantifié en 4 bits

poids principaux
→ gelés

adaptateurs LoRA
→ entraînaibles

Seulement environ 1,182 % des paramètres sont entraînaibles.

Cela rend l'expérience compatible avec une Tesla T4 et réduit fortement les besoins mémoire.

25 26. Technologies utilisées

```
Python
PyTorch
Transformers
Datasets
PEFT
TRL
bitsandbytes
rouge-score
Pandas
Gradio
Jupyter / Google Colab
VS Code
Git / GitHub
```

Rôle de chaque bibliothèque :

- torch : tenseurs et GPU ;
- transformers : modèle Qwen et tokenizer ;

- datasets : gestion des splits ;
 - peft : adaptateurs LoRA ;
 - bitsandbytes : quantification 4 bits ;
 - trl : SFTTrainer ;
 - rouge-score : métrique ROUGE-L ;
 - pandas : tableaux de résultats ;
 - gradio : interface utilisateur.
-

26 27. Ce que chaque fichier apporte

26.1 config.py

Centralise :

```
MODEL_ID
SEED
SYSTEM_PROMPT
```

But : ne pas répéter la configuration dans tous les fichiers.

26.2 model.py

Centralise :

```
chargement tokenizer
chargement modèle de base
chargement modèle quantifié
chargement adaptateur final
```

But : séparer la gestion des modèles de la logique d'évaluation.

26.3 generation.py

Centralise :

```
chat template
generate()
greedy
sampling
temperature
top-k
top-p
```

But : utiliser les mêmes règles de génération pendant toute l'expérience.

26.4 dataset.py

Construit :

```
concepts
questions
références
mots-clés
prompts
completions
splits train / validation / test
```

But : rendre les données reproductibles.

26.5 training.py

Contient :

```
BitsAndBytesConfig
LoraConfig
SFTConfig
SFTTrainer
```

But : isoler tous les choix liés au fine-tuning.

26.6 evaluation.py

Contient :

```
normalisation du texte
ROUGE-L
keyword coverage
évaluation du dataset
résumés
sauvegarde CSV
```

But : garantir que la baseline et QLoRA utilisent exactement les mêmes métriques.

26.7 app/app.py

Charge le modèle final et crée l'interface Gradio.

But : séparer l'application de démonstration du notebook d'entraînement.

26.8 notebooks/agent.ipynb

Le notebook relie toutes les briques :

```
installation
→ vérification GPU
→ tokenisation
→ logits
→ génération
→ baseline
→ dataset
→ QLoRA
→ entraînement
→ évaluation
→ challenge
→ Gradio
```

C'est le support principal de l'expérience.

27 28. Difficultés rencontrées pendant le développement

Plusieurs problèmes techniques ont dû être résolus :

27.1 Environnement Colab / VS Code

Le code local et le runtime distant ne représentent pas nécessairement le même système de fichiers.

Exemple :

```
/content/nlp_tutor
```

correspond au runtime Colab, et non directement au disque local.

27.2 Imports du package

Le projet utilise une structure src/ :

```
src/nlp_tutor/
```

Il a donc fallu s'assurer que Python utilise bien ce chemin.

27.3 Dépendances

Certaines bibliothèques ont dû être ajoutées :

```
trl  
bitsandbytes  
rouge-score
```

27.4 Mémoire GPU

Le projet devait tenir sur une Tesla T4.

La quantification 4 bits, LoRA, l'accumulation de gradients et le gradient checkpointing ont permis de limiter l'utilisation mémoire.

27.5 Sauvegarde

Le runtime Colab étant temporaire, les résultats et l'adaptateur ont été exportés vers un ZIP / Google Drive afin d'éviter de refaire l'entraînement.

28 29. Limites du projet

Les principales limites sont :

1. dataset synthétique relativement petit ;
 2. réponses de référence très similaires pour les reformulations d'un même concept ;
 3. ROUGE-L fortement favorisé par cette structure ;
 4. généralisation imparfaite sur certaines questions inédites ;
 5. historique conversationnel non exploité par le modèle ;
 6. domaine limité aux notions NLP choisies ;
 7. absence d'évaluation humaine formelle complète avec plusieurs évaluateurs.
-

29 30. Améliorations possibles

29.1 Données

Créer davantage de réponses différentes pour un même concept.

Ajouter :

comparaisons
cas pièges
questions multi-étapes
demandes hors périmètre
exemples négatifs

29.2 Évaluation

Construire un vrai jeu de test indépendant, rédigé séparément du dataset d'entraînement.
Ajouter une grille humaine :

exactitude
pertinence
clarté
respect du rôle
hallucinations

29.3 Conversation

Réinjecter un historique limité dans le prompt.

29.4 Déploiement

Déployer l'application sur un environnement stable plutôt qu'avec un lien temporaire Gradio.

30 31. Conclusion

Le projet a permis de parcourir tout le cycle de développement d'un petit assistant basé sur un Transformer :

comprendre le modèle
→ construire une baseline
→ préparer des données
→ adapter le modèle
→ mesurer les performances
→ analyser les limites
→ construire une interface

Le résultat quantitatif principal est :

ROUGE-L moyen
0,1331 → 1,0000

Couverture mots-clés
0,2500 → 0,8375

QLoRA a donc fortement spécialisé Qwen2.5-1.5B-Instruct sur le format pédagogique du dataset avec seulement environ 1,182 % des paramètres entraînaibles.

Cependant, l'expérience montre aussi qu'un score automatique excellent sur un dataset proche des données d'apprentissage ne doit pas être confondu avec une généralisation parfaite.

Le challenge sur de nouvelles formulations et l'analyse des réponses sont donc indispensables pour interpréter correctement les résultats.

31 32. Résumé en une phrase

NLP Tutor montre comment partir d'un petit Transformer pré-entraîné, observer son fonctionnement, créer une baseline, le spécialiser efficacement avec QLoRA, l'évaluer avant/après et l'intégrer dans une interface Gradio.